BTS SIO — Option SISR

Travaux Pratiques Linux / Administration Système

Révision / LAB 1

Commandes fondamentales Linux

Système · Fichiers · Répertoires · Liens · Navigation

Durée estimée	2h – 2h30
Objectif	Maîtriser les commandes de base du shell Linux
OS cible	Ubuntu 24.04 LTS

Pensez à prendre des captures d'écran et d'ajouter des explications afin de simplifier l'intégration sur votre portfolio.

Introduction au shell Linux

Qu'est-ce que le shell ? Le shell est l'interface en ligne de commande (CLI) de Linux.
Vous saisissez des commandes texte, le système les exécute et affiche le résultat.
C'est l'outil fondamental de tout administrateur système Linux.

Anatomie d'une commande Linux

```
# Structure générale :
$ commande   [options]   [arguments]

# Exemple décomposé :
$ ls         -l          /home/student
#  ↑          ↑             ↑
#  commande  option        argument

# Le symbole $ indique l'invite de commande (prompt) utilisateur
# Ne pas le taper ! Il représente "le terminal attend votre commande"
```

Symbole	Signification
$	Invite de commande utilisateur standard (ne pas taper)
#	Invite root (super-administrateur) OU commentaire dans le code
~	Raccourci pour votre répertoire personnel

Symbole	Signification
	(/home/votre_login)
..	Répertoire parent (niveau au-dessus)
.	Répertoire courant

Astuce :
- Utilisez la touche Tab pour auto-compléter les noms de commandes et de fichiers.
- Utilisez les flèches ↑↓ pour naviguer dans l'historique des commandes.

Partie A — Informations système et date

1 Obtenir des informations sur le système

Cette tâche vous apprend à interroger votre système Linux :
- quel OS tourne,
- quel est votre identifiant,
- quel est le nom de la machine.

(i) Afficher le nom du système d'exploitation

```
# uname affiche des informations sur le noyau et le système
$ uname -s

# Sortie attendue :
Linux

# Pour plus de détails (version complète du noyau)
$ uname -a

# Nom de la distribution (Ubuntu, Debian...)
$ cat /etc/os-release
$ lsb_release -a
```

```
etudinfo@pc-etudinfo:~$ uname -s
Linux
etudinfo@pc-etudinfo:~$ uname -a
Linux pc-etudinfo 7.0.0-30-generic #30-Ubuntu SMP PREEMPT_DYNAMIC Fri Jul 31 18:
22:54 UTC 2026 x86_64 GNU/Linux
etudinfo@pc-etudinfo:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04.1 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-poli
cy"
```

Option uname	Affiche
-s	Nom du noyau (Linux)
-r	Version du noyau
-m	Architecture processeur (x86_64, arm64...)
-a	Toutes les informations ci-dessus

(ii) Afficher le nom de connexion (login)

> whoami signifie littéralement "Qui suis-je ?"
> Il retourne le nom de l'utilisateur actuellement connecté dans le terminal.

```
# Nom de l'utilisateur courant
$ whoami

# Sortie attendue :
etudinfo

# Informations complètes sur l'utilisateur (UID, groupes...)
$ id

# Sortie attendue :
uid=1000(etudinfo) gid=1000(etudiant) groups=1000(etudiant),27(sudo)
```

```
etudinfo@pc-etudinfo:~$ whoami
etudinfo
```

```
etudinfo@pc-etudinfo:~$ id
uid=1000(etudinfo) gid=1000(etudinfo) groups=1000(etudinfo),4(adm),24(cdrom),
27(sudo),30(dip),46(plugdev),100(users),111(lpadmin),114(lxd)
```

(iii) Afficher le nom de la machine (hostname)

> Le hostname est le "nom" donné à la machine sur le réseau. Comme un prénom pour l'ordinateur.

```
# Nom de la machine
$ hostname

# Sortie attendue :
travel-horizon-srv

# Nom de domaine complet (FQDN)
$ hostname -f
```

```
etudinfo@pc-etudinfo:~$ hostname
pc-etudinfo
etudinfo@pc-etudinfo:~$ hostname -f
pc-etudinfo
```

2 Afficher les utilisateurs connectés

> Ces commandes permettent de savoir qui est connecté sur le système en ce moment.
> Très utile pour un administrateur qui surveille un serveur.

```
# Tous les utilisateurs actuellement connectés
$ who

# Sortie attendue :
Etudinfo pts/0  2026-09-15 09:12 (:0)
alice    pts/1  2026-09-15 09:45 (192.168.1.10)
```

```
# Version plus détaillée (heure de connexion, activité...)
$ w

# Chercher un utilisateur particulier (ex: alice)
$ who | grep alice

# Votre propre nom de session
$ who am i
```

```
etudinfo@pc-etudinfo:~$ w
 12:40:08 up 14 min,  1 user,  load average: 0,53, 0,36, 0,24
UTIL.    TTY      DE               LOGIN@   IDLE   JCPU   PCPU  WHAT
etudinfo tty2     -                12:25   14:24   0.01s  0.01s /usr/libexec
```

Commande	Informations affichées
who	Utilisateurs connectés, terminal, heure de connexion
w	Idem + commande en cours, charge CPU
who | grep nom	Vérifie si un utilisateur précis est connecté
who am i	Votre propre session uniquement
last	Historique des dernières connexions

3 Afficher le calendrier

La commande cal affiche un calendrier dans le terminal. Utile pour vérifier des dates sans quitter la ligne de commande.

(i) Calendrier de Janvier 2000

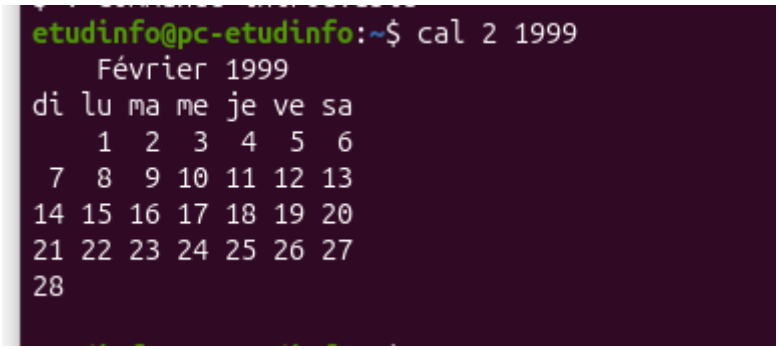
```
# cal [mois] [année]
$ cal 1 2000

# Sortie attendue :
    Janvier 2000
Di Lu Ma Me Je Ve Sa
                1
2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30 31
```

```
etudinfo@pc-etudinfo:~$ cal 1 2000
    Janvier 2000
di lu ma me je ve sa
                   1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30 31
```

(ii) Calendrier de Février 1999

```
$ cal 2 1999
```



(iii) 9e mois de l'an 7 après J.-C.

```
# Septembre de l'an 7
$ cal 9 7
```

```
etudinfo@pc-etudinfo:~$ cal 9 7
    Septembre 7
di lu ma me je ve sa
             1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30
```

(iv) Mois courant

```
# Sans argument = mois courant
$ cal
```

```
etudinfo@pc-etudinfo:~$ cal
   Septembre 2026
di lu ma me je ve sa
       1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30
```

(v) Date abrégée : jour, mois, année

```
# date avec format personnalisé
# %a = abréviation du jour  %b = abréviation du mois  %Y = année
$ date +"%a %b %Y"

# Sortie attendue (exemple) :
Dim Sep 2026
```

```
etudinfo@pc-etudinfo:~$ date +"%a %b %Y"
ven. sept 2026
```

4 Afficher l'heure en format 12h et 24h

La commande date affiche la date ET l'heure. On peut la formater avec des codes :

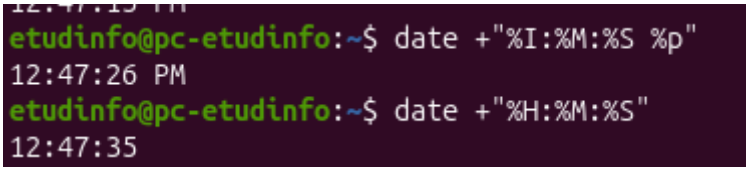
%I = heure en format 12h (01-12) %H = heure en format 24h (00-23)
%M = minutes %S = secondes %p = AM ou PM

```
# Format 12 heures (avec AM/PM)
$ date +"%I:%M:%S %p"

# Sortie attendue :
09:35:42 AM

# Format 24 heures
$ date +"%H:%M:%S"

# Sortie attendue :
09:35:42
```



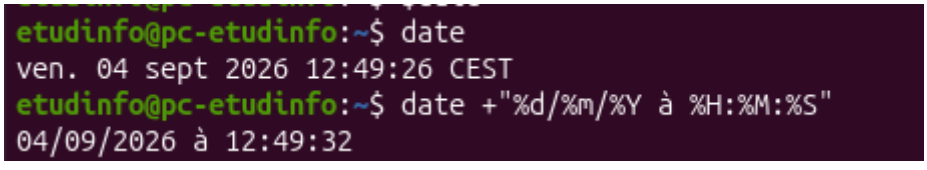
5 Afficher la date et l'heure courantes

```
# Date et heure complètes
$ date

# Sortie attendue :
dim. 15 sept. 2026 09:35:42 CEST

# Format personnalisé complet
$ date +"%d/%m/%Y à %H:%M:%S"

# Sortie attendue :
15/09/2026 à 09:35:42
```



6 Afficher "GOOD MORNING" en grands caractères

La commande figlet génère du texte en "ASCII art" — des grandes lettres formées par des

caractères. Très utilisée pour les messages d'accueil sur les serveurs.

```
# Installer figlet si nécessaire
$ sudo apt install figlet -y

# Afficher GOOD MORNING en grands caractères
$ figlet "GOOD MORNING"

# Sortie attendue :
 ____  ___   ___  ____    __  __  ___  ____  _   _  ___  _   _  ____
/ ___)/ _ \ / _ \|  _ \  |  \/  |/ _ \|  _ \| \ | ||_ _|| \ | |/ ___)

# Avec une autre police
$ figlet -f slant "GOOD MORNING"

# Alternative avec banner (plus simple)
$ sudo apt install sysvbanner -y
$ banner "GOOD MORNING"
```

```
etudinfo@pc-etudinfo:~$ figlet "GOOD MORNING"
  ____  ___   ___  ____    __  __  ___  ____  _   _  ___  _   _  ____
 / ___|/ _ \ / _ \|  _ \  |  \/  |/ _ \|  _ \| \ | ||_ _|| \ | |/ ___|
| |  _| | | | | | | | | | | |\/| | | | | |_) |  \| | | | |  \| | |  _
| |_| | |_| | |_| | |_| | | |  | | |_| |  _ <| |\  | | | | |\  | |_| |
 \____|\___/ \___/|____/  |_|  |_|\___/|_| \_\_| \_||___||_| \_|\____|
```

Script — Partie A

1. Les administrateurs systèmes aiment personnaliser leur environnement de travail. Un script de bienvenue (ou "motd" - Message Of The Day) affiche automatiquement des informations pertinentes dès l'ouverture du terminal : date, heure, utilisateur, machine, etc.

Votre mission : créer un tel script !

Vous décrivez toutes les étapes de réalisation du script

```
==================================================
              🖥  BIENVENUE SUR LINUX  🖥
==================================================

📌 INFORMATIONS SYSTÈME
-----------------------
  OS            : Linux
  Noyau         : 6.8.0-45-generic
  Architecture  : x86_64
  Version       : #45-Ubuntu

📌 DISTRIBUTION
---------------
  Ubuntu 24.04.1 LTS

📌 UTILISATEUR
--------------
  Login         : etudinfo
  UID/GID       : uid=1000(etudinfo)
  Groupes       : 1000(etudiant),27(sudo)

📌 MACHINE
----------
  Hostname      : travel-horizon-srv
  FQDN          : travel-horizon-srv.local
```

```
📌 DATE ET HEURE
----------------
  Date complète : dim. 15 sept. 2026 09:35:42 CEST
  Format 12h    : 09:35:42 AM
  Format 24h    : 09:35:42
  Calendrier    : Sunday 15 September 2026

📌 CALENDRIER DU MOIS
---------------------
    Septembre 2026
Di Lu Ma Me Je Ve Sa
        1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30

📌 UTILISATEURS CONNECTÉS
-------------------------
  etudinfo pts/0        2026-09-15 09:12 (:0)
  alice    pts/1        2026-09-15 09:45 (192.168.1.10)
```

#!/bin/bash

--- INFORMATIONS SYSTÈME ---

```
OS=$(uname -s) NOYAU=$(uname -r) ARCH=$(uname -m) VERSION=$(uname -v | awk '{print $1}')
```

--- DISTRIBUTION ---

```
DISTRO=$(lsb_release -d 2>/dev/null | cut -f2 || cat /etc/os-release | grep PRETTY_NAME | cut -d"" -f2)
```

--- UTILISATEUR ---

```
LOGIN=$(whoami) UID_GID=$(id -u):$(id -g) GROUPES=$(id -Gn | tr ' ' ',')
```

--- MACHINE ---

```
HOSTNAME_NAME=$(hostname) FQDN=$(hostname -f 2>/dev/null || echo "$HOSTNAME_NAME")
```

--- DATE ET HEURE ---

```
DATE_COMPLETE=$(date +"%a. %d %b. %Y %H:%M:%S %Z") FORMAT_12H=$(date +"%I:%M:%S %p") FORMAT_24H=$(date +"%H:%M:%S") CALENDRIER_DATE=$(date +"%A %d %B %Y")
```

--- AFFICHAGE ---

```
echo "================================================" echo "        💻 BIENVENUE SUR LINUX 💻        " echo "================================================" echo ""

echo "📌 INFORMATIONS SYSTÈME" echo "----------------------" echo "OS          : $OS" echo "Noyau       : $NOYAU" echo "Architecture : $ARCH" echo "Version     : $VERSION" echo ""

echo "📌 DISTRIBUTION" echo "----------------------" echo "$DISTRO" echo ""

echo "📌 UTILISATEUR" echo "----------------------" echo "Login       : $LOGIN" echo "UID/GID     : uid=$(id -u)($LOGIN)" echo "Groupes     : $(id -G | sed 's/ /(/g' | sed 's/$/)/')" # Format type: 1000(etudiant),27(sudo) echo ""

echo "📌 MACHINE" echo "----------------------" echo "Hostname    : $HOSTNAME_NAME" echo "FQDN        : $FQDN" echo ""

echo "📌 DATE ET HEURE" echo "----------------------" echo "Date complète : $DATE_COMPLETE" echo "Format 12h    : $FORMAT_12H" echo "Format 24h    : $FORMAT_24H" echo "Calendrier    : $CALENDRIER_DATE" echo ""

echo "📌 CALENDRIER DU MOIS" echo "----------------------" cal echo ""
```

```
================================================
          💻   BIENVENUE SUR LINUX   💻
================================================

📌 INFORMATIONS SYSTÈME
----------------------
OS           : Linux
Noyau        : 7.0.0-30-generic
Architecture : x86_64
Version      : #30-Ubuntu

📌 DISTRIBUTION
----------------------
Ubuntu 26.04.1 LTS

📌 UTILISATEUR
----------------------
Login        : etudinfo
UID/GID      : uid=1000(etudinfo)
Groupes      : 1000(4(24(27(30(46(100(111(114)

📌 MACHINE
----------------------
Hostname     : pc-etudinfo
FQDN         : pc-etudinfo

📌 DATE ET HEURE
----------------------
Date complète : ven.. 04 sept. 2026 12:54:04 CEST
Format 12h    : 12:54:04 PM
Format 24h    : 12:54:04
Calendrier    : vendredi 04 septembre 2026

📌 CALENDRIER DU MOIS
----------------------
   Septembre 2026
di lu ma me je ve sa
       1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30


📌 UTILISATEURS CONNECTÉS
----------------------
```

Partie B — Navigation et gestion des répertoires

Comprendre l'arborescence Linux avant de commencer :
/ → Racine du système (root de l'arbre)
/home/student → Votre répertoire personnel (home directory)
/home/student/SAMPLE → sous-répertoire SAMPLE
/home/student/SAMPLE/TRIAL → sous-sous-répertoire TRIAL
Règle : les chemins absolus commencent par / | les chemins relatifs commencent depuis le dossier courant

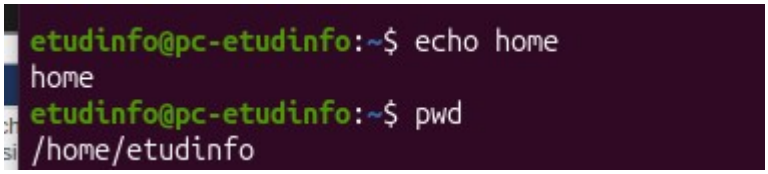
7 Afficher le répertoire personnel (home)

```
# echo affiche du texte ou la valeur d'une variable
# $HOME est une variable d'environnement = votre chemin home
$ echo $HOME

# Sortie attendue :
/home/student

# Alternative : pwd depuis le home
$ cd ~
$ pwd

# Sortie attendue :
/home/student
```



Commande / Variable	Signification
echo $HOME	Affiche la valeur de la variable HOME (votre dossier personnel)
~	Raccourci clavier équivalent à $HOME dans les commandes
pwd	Print Working Directory = affiche le chemin du répertoire courant

8 Créer le répertoire SAMPLE dans le home

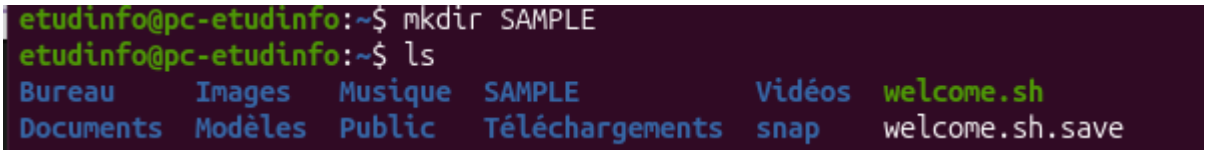
mkdir = Make Directory. C'est la commande pour créer un nouveau dossier.

```
# Aller dans le home d'abord
$ cd ~

# Créer le répertoire SAMPLE
$ mkdir SAMPLE


# Vérifier qu'il existe
$ ls

# Sortie attendue (SAMPLE visible) :
Desktop  Documents  Downloads  SAMPLE
```



9 Créer le sous-répertoire TRIAL dans SAMPLE

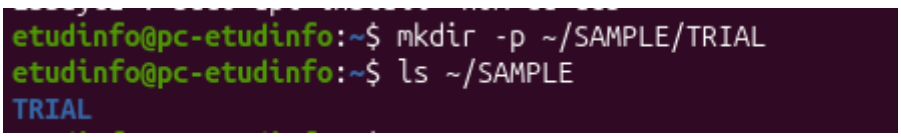
```
# Méthode 1 : entrer dans SAMPLE puis créer TRIAL
$ cd ~/SAMPLE
$ mkdir TRIAL

# Méthode 2 : en une seule commande depuis n'importe où
$ mkdir ~/SAMPLE/TRIAL

# Méthode 3 : créer toute la chaîne d'un coup (-p = parents)
$ mkdir -p ~/SAMPLE/TRIAL

# Vérifier la structure
$ ls ~/SAMPLE

# Sortie attendue :
TRIAL
```



L'option -p de mkdir crée tous les répertoires parents nécessaires. Exemple : mkdir -p /a/b/c/d crée a, puis b, puis c, puis d en une seule commande.

10 Se déplacer dans SAMPLE

cd = Change Directory. C'est la commande pour naviguer dans l'arborescence.

```
# Chemin absolu (commence depuis la racine /)
$ cd /home/student/SAMPLE

# Chemin relatif (depuis le home ~)
$ cd ~/SAMPLE

# Vérifier où on est
$ pwd

# Sortie attendue :
/home/student/SAMPLE
```

```
etudinfo@pc-etudinfo:~$ mkdir -p ~/SAMPLE/TRIAL
etudinfo@pc-etudinfo:~$ ls ~/SAMPLE
TRIAL
```

11 Revenir au répertoire home

```
# Méthode 1 : cd sans argument → va toujours au home
$ cd

# Méthode 2 : avec le raccourci ~
$ cd ~

# Méthode 3 : avec la variable $HOME
$ cd $HOME

# Vérifier
$ pwd
# Sortie : /home/student
```

```
etudinfo@pc-etudinfo:~/SAMPLE$ cd $HOME
etudinfo@pc-etudinfo:~$ pwd
/home/etudinfo
```

12 Aller dans TRIAL depuis le home — chemin absolu ET relatif

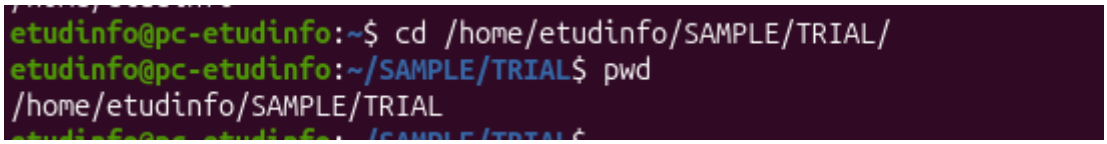
Chemin ABSOLU : part toujours de la racine /, indépendant de votre position actuelle.
Chemin RELATIF : part de votre position actuelle, utilise . (ici) et .. (dossier parent).

```
# ── Chemin absolu ──────────────────────────────────────────
# Commence par / , fonctionne depuis n'importe où
$ cd /home/student/SAMPLE/TRIAL
$ pwd
# Sortie : /home/student/SAMPLE/TRIAL

# ── Chemin relatif ─────────────────────────────────────────
# D'abord revenir au home
$ cd ~

# Puis naviguer relativement
$ cd SAMPLE/TRIAL
$ pwd
# Sortie : /home/student/SAMPLE/TRIAL

# ── Remonter avec .. ───────────────────────────────────────
# .. = répertoire parent
$ cd ..         # remonte dans SAMPLE
$ cd ../..      # remonte deux niveaux (dans /home/student)
```



13 Créer le répertoire TEST par chemin absolu

```
# Depuis n'importe quel répertoire courant,
# un chemin absolu crée le dossier exactement où indiqué
$ mkdir /home/student/TEST

# Ou avec ~
$ mkdir ~/TEST

# Vérifier
$ ls ~
# Sortie attendue :
SAMPLE  TEST  Desktop  Documents ...
```

```
etudinfo@pc-etudinfo:~$ mkdir -p  /home/etudinfo/student/test
etudinfo@pc-etudinfo:~$ ls
Bureau     Modèles  SAMPLE           snap         welcome.sh.save
Documents  Musique  Téléchargements  student
Images     Public   Vidéos           welcome.sh
```

14 Revenir au home en une seule commande

```
# cd sans argument = raccourci universel vers $HOME
$ cd

# Vérifier
$ pwd
# Sortie : /home/student
```

```
etudinfo@pc-etudinfo:~$ cd /home/etudinfo/SAMPLE/TRIAL/
etudinfo@pc-etudinfo:~/SAMPLE/TRIAL$ cd
etudinfo@pc-etudinfo:~$ pwd
/home/etudinfo
```

15 Supprimer un répertoire par chemin absolu

> rmdir = Remove Directory. Ne fonctionne que sur un répertoire VIDE.
> Pour supprimer un répertoire avec des fichiers : rm -rf (voir Tâche 31).

```
# Supprimer le répertoire TEST (vide)
$ rmdir /home/student/TEST

# Vérifier
$ ls ~
# TEST ne doit plus apparaître

# Si TEST contient des fichiers, rmdir échouera :
# rmdir: failed to remove '/home/student/TEST': Directory not empty
```

```
etudinfo@pc-etudinfo:~$ rmdir /home/etudinfo/SAMPLE/TRIAL/
etudinfo@pc-etudinfo:~$ ls
Bureau     Modèles  SAMPLE           snap         welcome.sh.save
Documents  Musique  Téléchargements  student
Images     Public   Vidéos           welcome.sh
```

Questions de compréhension — Partie B

1. Quelle différence entre cd ~ et cd .. ?
2. Depuis /home/student/SAMPLE/TRIAL, quel chemin relatif permet d'aller dans /home/student ?
3. Pourquoi rmdir échoue-t-il si le répertoire contient des fichiers ?
4. Que fait la commande mkdir -p a/b/c/d si le dossier b existe déjà ?

1)`cd` amènedirectement à ton répertoire personnel (home), peu importe où tu es dans l'arborescence, c'est un chemin absolu.
Cd .. fait remonter d'un seul niveau, vers le répertoire parent de l'endroit où tu es actuellement c'est un chemin relatif.

2) c'est cd ..

3) `rmdir` est volontairement limité à la suppression de répertoires vides, par sécurité, il faut utiliser rm -r

4) `mkdir -p a/b/c/d` si b existe déjà
Grâce à l'option `-p`, `mkdir` ne renvoie aucune erreur pour les dossiers déjà existants

Partie C — Création et gestion des fichiers

16 Créer les fichiers myfile et yourfile

Deux façons courantes de créer un fichier :
- touch : crée un fichier vide (ou met à jour la date si existe déjà)
- cat > fichier : crée un fichier et attend que vous tapiez du contenu (Ctrl+D pour finir)

```
# S'assurer d'être dans le bon répertoire
$ cd ~

# Méthode 1 : créer des fichiers vides avec touch
$ touch myfile yourfile

# Méthode 2 : créer avec contenu avec cat >
$ cat > myfile
Bonjour, ceci est myfile.
Ligne 2 de myfile.
# Appuyer sur Ctrl+D pour sauvegarder et quitter

$ cat > yourfile
Bonjour, ceci est yourfile.
Ligne 2 de yourfile.
# Ctrl+D

# Vérifier
$ ls -l myfile yourfile
```

```
etudinfo@pc-etudinfo:~$ cd
etudinfo@pc-etudinfo:~$ touch myfile yourfile
etudinfo@pc-etudinfo:~$ cat > myfile
Bonjour, ceci est myfile
```

```
etudinfo@pc-etudinfo:~$ cat yourfile
Bonjour, ceci est yourfile
Ligne 2
```

17 Afficher le contenu des fichiers

cat = Concatenate. Affiche le contenu d'un fichier dans le terminal. C'est l'un des outils les plus utilisés en administration Linux.

```
# Afficher myfile
```

```
$ cat myfile

# Afficher yourfile
$ cat yourfile

# Afficher les deux à la suite
$ cat myfile yourfile

# Afficher avec numéros de ligne
$ cat -n myfile

# Afficher par pages (si fichier long)
$ less myfile
# Naviguer : espace = page suivante, q = quitter
```

18 Ajouter des lignes dans myfile et yourfile

Attention à la différence :
> (redirection simple) : ÉCRASE le fichier existant
>> (redirection double) : AJOUTE à la fin (append) sans effacer

```
# Ajouter des lignes à myfile (>> = append)
$ cat >> myfile
Troisième ligne ajoutée.
Quatrième ligne ajoutée.
# Ctrl+D

# Ajouter à yourfile
$ cat >> yourfile
Nouvelle ligne dans yourfile.
# Ctrl+D

# Vérifier le contenu après ajout
$ cat myfile

# Alternative avec echo >>
$ echo "Une ligne supplémentaire" >> myfile
```

19 Créer un fichier caché

Sous Linux, tout fichier dont le nom commence par un point (.) est automatiquement caché. Il n'apparaît pas avec ls ordinaire, mais avec ls -a.

```
# Créer un fichier caché (préfixe .)
```

```
$ touch .fichier_cache

# ls ordinaire : ne le voit pas
$ ls

# ls -a (all) : voit les fichiers cachés
$ ls -a

# Sortie attendue :
.  ..  .fichier_cache  myfile  yourfile

# ls -la : caché + détails
$ ls -la
```

Les fichiers de configuration Linux sont souvent cachés : .bashrc, .profile, .ssh/, .gitconfig... C'est une convention Unix, pas une vraie protection de sécurité.

20 Copier myfile vers emp

cp = Copy. Copie un fichier ou répertoire. La source reste intacte, une nouvelle copie est créée.

```
# cp source destination
$ cp myfile emp

# Vérifier
$ ls
# myfile et emp existent tous les deux

# Afficher emp pour confirmer le contenu copié
$ cat emp

# cp avec confirmation si écrasement (-i = interactive)
$ cp -i myfile emp
```

21 Créer un alias (lien symbolique) pour un fichier

Un lien symbolique (alias) est un raccourci qui pointe vers un autre fichier.
ln -s source lien → crée un alias "lien" qui pointe vers "source"
Modifier l'alias modifie l'original. Supprimer l'alias ne supprime pas l'original.

```
# ln -s = symbolic link (lien symbolique = alias)
# ln -s fichier_original nom_du_lien
$ ln -s myfile alias_myfile
```

```
# Vérifier : ls -l montre la flèche ->
$ ls -l alias_myfile

# Sortie attendue :
lrwxrwxrwx 1 student student 6 sept. 15 09:00 alias_myfile -> myfile

# Le contenu est identique
$ cat alias_myfile
```

Type de lien	Commande	Caractéristique principale
Lien symbolique (souple)	ln -s source lien	Raccourci (comme un alias) — peut pointer vers répertoires
Lien physique (dur)	ln source lien	Même inode que l'original — survit à la suppression de l'original

22 Déplacer yourfile vers dept

mv = Move. Déplace OU renomme un fichier. Contrairement à cp, la source disparaît.

```
# mv source destination
$ mv yourfile dept

# Vérifier : yourfile n'existe plus
$ ls
# Sortie attendue : dept  emp  myfile (plus yourfile)

# Afficher le contenu de dept
$ cat dept
# Contenu identique à yourfile
```

mv sert aussi à renommer : mv ancien_nom nouveau_nom dans le même répertoire = renommage.

23 Copier emp et dept dans le répertoire TRIAL

```
# Copier deux fichiers vers un répertoire
$ cp emp dept ~/SAMPLE/TRIAL/

# Vérifier dans TRIAL
$ ls ~/SAMPLE/TRIAL/
```

```
# Sortie attendue :
dept  emp
```

24 Comparer des fichiers avec diff

diff compare deux fichiers ligne par ligne et affiche les différences. Si les fichiers sont identiques, aucune sortie n'est produite.

```
# ── Tâche 25 : comparer un fichier avec lui-même ──────────────
$ diff myfile myfile
# Aucune sortie = fichiers identiques

# ── Tâche 26 : comparer myfile et emp ─────────────────────────
$ diff myfile emp
# Aucune sortie attendue (emp est une copie de myfile)

# Lecture de la sortie diff (si différences) :
# < = ligne dans le 1er fichier
# > = ligne dans le 2e fichier
# --- = séparateur

# Exemple de sortie avec différences :
# 3c3
# < Troisième ligne de myfile
# ---
# > Troisième ligne de emp (différente)
```

25 Ajouter deux lignes dans emp dans TRIAL

```
# Ajouter des lignes à emp qui est DANS TRIAL
$ cat >> ~/SAMPLE/TRIAL/emp
Ligne ajoutée directement dans TRIAL/emp.
Deuxième ligne ajoutée.
# Ctrl+D

# Vérifier
$ cat ~/SAMPLE/TRIAL/emp
```

26 Comparer les fichiers emp du home et de TRIAL

```
# Tâche 28 : comparer emp du home avec emp dans TRIAL
$ diff ~/emp ~/SAMPLE/TRIAL/emp
```

```
# Sortie attendue (les 2 lignes ajoutées apparaissent) :
3a4,5
> Ligne ajoutée directement dans TRIAL/emp.
> Deuxième ligne ajoutée.

# Tâche 29 : diff avec sortie formatée pour meilleure lisibilité
$ diff -u ~/emp ~/SAMPLE/TRIAL/emp

# -u = format unifié (plus lisible) :
# --- emp (version home)
# +++ TRIAL/emp (version TRIAL)
# -ligne supprimée  +ligne ajoutée
```

Questions de compréhension — Partie C

1. Quelle différence entre > et >> dans une redirection ?
2. Que se passe-t-il si on utilise cp sans l'option -i sur un fichier existant ?
3. mv yourfile dept : yourfile existe-t-il encore après cette commande ?
4. Comment créer un fichier caché nommé .secret ?

1) > **écrase** le fichier de destination : si le fichier existe déjà, son contenu est remplacé entièrement par ce qui est redirigé. S'il n'existe pas, il est créé.
>> **ajoute à la suite** (append) : le contenu redirigé est ajouté à la fin du fichier existant, sans toucher à ce qu'il y avait déjà. S'il n'existe pas, il est créé aussi.

2) **cp sans `-i` sur un fichier existant**
Le fichier de destination est **écrasé silencieusement**, sans aucune confirmation ni avertissement — la copie remplace le contenu existant directement. C'est pour ça que `-i` (interactive) est recommandé : il demande confirmation avant d'écraser.

3) Non. `mv` **déplace** (ou renomme) le fichier, il ne le copie pas. Après la commande, `yourfile` n'existe plus à son emplacement d'origine — il se trouve désormais dans `dept` (soit renommé en `dept` si c'est un nom de fichier, soit déplacé dans le dossier `dept` s'il s'agit d'un répertoire existant).

Touch .fichier
Sur linux, les nom qui commencent par un point sont cachés.

Partie D — Suppression, renommage et liens

27 Supprimer les fichiers dans TRIAL

rm = Remove. Supprime des fichiers. ATTENTION : sous Linux, la suppression est définitive, il n'y a pas de corbeille en ligne de commande !

```
# Supprimer un fichier spécifique dans TRIAL
$ rm ~/SAMPLE/TRIAL/emp

# Supprimer plusieurs fichiers
$ rm ~/SAMPLE/TRIAL/emp ~/SAMPLE/TRIAL/dept

# Supprimer avec confirmation (-i = interactive)
$ rm -i ~/SAMPLE/TRIAL/emp
# Le système demande : remove 'emp'? → taper y puis Entrée

# Vérifier
$ ls ~/SAMPLE/TRIAL/
# Répertoire vide attendu
```

28 Supprimer un répertoire avec des fichiers en une seule commande

rm -rf = Remove Recursive Force
-r = récursif (supprime le contenu et les sous-dossiers)
-f = force (pas de confirmation, pas d'erreur si absent)
DANGER : rm -rf /* supprimerait tout le système ! Toujours vérifier avant d'exécuter.

```
# rmdir échoue si le répertoire n'est pas vide :
# rmdir: failed to remove 'TRIAL': Directory not empty

# rm -r supprime un répertoire et tout son contenu
$ rm -r ~/SAMPLE/TRIAL

# rm -rf : sans demande de confirmation
$ rm -rf ~/SAMPLE/TRIAL

# Bonne pratique : toujours lister avant de supprimer
$ ls ~/SAMPLE/TRIAL   # vérifier ce qu'on va supprimer
$ rm -rf ~/SAMPLE/TRIAL
```

29 Peut-on récupérer un fichier supprimé ?

Réponse courte : NON, pas facilement. Sous Linux en ligne de commande :
- rm supprime définitivement (pas de corbeille)
- Récupération possible via des outils forensiques (testdisk, extundelete) mais complexe
- La règle d'or : faire des SAUVEGARDES régulières (tar, rsync, Timeshift)

```
# Il n'existe pas de commande "undelete" native sous Linux

# Outil de récupération forensique (si filesystem ext4)
$ sudo apt install extundelete -y
$ sudo extundelete /dev/sda1 --restore-file home/student/emp

# Outil plus général
$ sudo apt install testdisk -y
$ sudo testdisk

# PRÉVENTION : créer une sauvegarde AVANT de supprimer
$ cp emp emp.backup
$ tar czf sauvegarde.tar.gz ~/SAMPLE/
```

30 Renommer TRIAL en DATA

Pour renommer un répertoire sous Linux, on utilise mv (même commande que pour déplacer). mv ancien_nom nouveau_nom dans le même dossier = renommage.

```
# Recréer TRIAL pour cet exercice
$ mkdir ~/SAMPLE/TRIAL

# Renommer TRIAL en DATA
$ mv ~/SAMPLE/TRIAL ~/SAMPLE/DATA

# Vérifier
$ ls ~/SAMPLE/
# Sortie attendue :
DATA

# TRIAL n'existe plus, DATA existe
```

31 Copier DATA vers un nouveau répertoire TRIAL

Pour copier un répertoire entier avec cp, l'option -r (récursif) est obligatoire. Sans -r, cp ne copie pas les dossiers.

```
# cp -r = copie récursive (répertoire + contenu)
$ cp -r ~/SAMPLE/DATA ~/SAMPLE/TRIAL

# Vérifier
$ ls ~/SAMPLE/
# Sortie attendue :
DATA  TRIAL

# DATA et TRIAL existent tous les deux
```

32 Liens physiques et symboliques

Rappel des deux types de liens :
ln -s source lien → Lien SYMBOLIQUE (souple) : comme un raccourci Windows
ln source lien → Lien PHYSIQUE (dur) : deux noms pour le même fichier

Créer dummy dans TRIAL et le lier à star

```
# Créer le fichier dummy dans TRIAL
$ cat > ~/SAMPLE/TRIAL/dummy
Contenu du fichier dummy.
# Ctrl+D

# Créer un lien symbolique star qui pointe vers dummy
$ ln -s ~/SAMPLE/TRIAL/dummy ~/SAMPLE/TRIAL/star

# Vérifier
$ ls -l ~/SAMPLE/TRIAL/

# Sortie attendue :
-rw-r--r-- 1 student student 26 sept. 15 dummy
lrwxrwxrwx 1 student student 30 sept. 15 star ->
/home/student/SAMPLE/TRIAL/dummy

# Lire via le lien = lire l'original
$ cat ~/SAMPLE/TRIAL/star
```

Lier dummy (TRIAL) à power dans DATA

```
# Lien PHYSIQUE de dummy (TRIAL) vers power (DATA)
$ ln ~/SAMPLE/TRIAL/dummy ~/SAMPLE/DATA/power

# Vérifier : ls -li montre le numéro d'inode
$ ls -li ~/SAMPLE/TRIAL/dummy ~/SAMPLE/DATA/power

# Sortie attendue (même inode = même fichier) :
```

```
123456 -rw-r--r-- 2 student student 26 ... /home/student/SAMPLE/DATA/power
123456 -rw-r--r-- 2 student student 26 ... /home/student/SAMPLE/TRIAL/dummy
# Le chiffre 2 = nombre de liens vers cet inode

# Modifier power modifie dummy et vice-versa
$ echo "Ajout via power" >> ~/SAMPLE/DATA/power
$ cat ~/SAMPLE/TRIAL/dummy
# La ligne ajoutée apparaît !
```

Critère	Lien symbolique (ln -s)	Lien physique (ln)
Commande	ln -s source lien	ln source lien
Fonctionne sur répertoires	✓ Oui	✗ Non (en général)
Si original supprimé	✗ Lien cassé (dangling)	✓ Fichier toujours accessible
Filesystems différents	✓ Oui	✗ Non (même FS obligatoire)
Visible dans ls -l	-> (flèche)	Même taille/inode

Synthèse — Tableau de bord des commandes

Commande	Syntaxe principale	Rôle	Partie
uname	uname -a	Informations sur le noyau et l'OS	A
whoami	whoami	Nom de l'utilisateur courant	A
hostname	hostname	Nom de la machine	A
who / w	who | w	Utilisateurs connectés	A
cal	cal [mois] [année]	Afficher un calendrier	A
date	date +"%format"	Date et heure (formatées)	A
figlet	figlet "texte"	Texte en gros caractères ASCII	A
echo	echo $VARIABLE	Afficher texte ou variable	B
pwd	pwd	Répertoire courant	B
cd	cd [chemin]	Changer de répertoire	B
mkdir	mkdir [-p] répertoire	Créer un répertoire	B
rmdir	rmdir répertoire	Supprimer répertoire vide	B
ls	ls [-la]	Lister les fichiers	B/C
touch	touch fichier	Créer fichier vide	C
cat	cat [>|>>] fichier	Créer / lire / ajouter dans fichier	C
cp	cp [-r] source dest	Copier fichier ou répertoire	C
mv	mv source dest	Déplacer ou renommer	C
diff	diff [-u] f1 f2	Comparer deux fichiers	C
rm	rm [-rf] fichier	Supprimer fichier ou répertoire	D
ln	ln source lien	Lien physique	D
ln -s	ln -s source lien	Lien symbolique (alias)	D

Aide-mémoire rapide — Navigation

```
# Raccourcis essentiels
$ cd           # → home
$ cd ~         # → home
$ cd ..        # → répertoire parent
$ cd -         # → répertoire précédent
$ pwd          # → où suis-je ?
$ ls -la       # → que contient ce dossier ? (cachés inclus)

# Chemins
# Absolu  : /home/student/SAMPLE/TRIAL  (commence par /)
# Relatif : SAMPLE/TRIAL                (depuis position actuelle)
# Tilde   : ~/SAMPLE/TRIAL              (depuis le home)
```

Récapitulatif des redirections

Symbole	Nom	Effet	Exemple
>	Redirection sortie	Crée ou ÉCRASE le fichier	echo "hello" > fichier
>>	Redirection append	AJOUTE à la fin du fichier	echo "hello" >> fichier
<	Redirection entrée	Lit depuis un fichier	cat < fichier
|	Pipe	Envoie la sortie vers une autre commande	who | grep alice

Compétences BTS SIO SISR mobilisées

Bloc de compétences	Activités couvertes
B2 — Administration des serveurs Linux	Navigation, gestion fichiers/répertoires, shell
B2 — Utilisation des commandes système	Informations système, date, utilisateurs, liens
B2 — Organisation du système de fichiers	Chemins absolus/relatifs, arborescence, liens
B3 — Documentation technique	Compte-rendu des commandes, tableaux de vérification

Mission d'auto-évaluation

Sans regarder ce document, répondez aux questions suivantes en tapant les commandes :

1. Quelle commande affiche le répertoire courant ?
2. Créer la structure : ~/PROJET/SRC/CODE en une seule commande
3. Créer un fichier readme.txt avec le contenu "Mon projet"
4. Copier readme.txt dans ~/PROJET/SRC/
5. Afficher les différences entre les deux copies

6. Créer un lien symbolique raccourci vers ~/PROJET/SRC/readme.txt
7. Supprimer ~/PROJET et tout son contenu en une seule commande